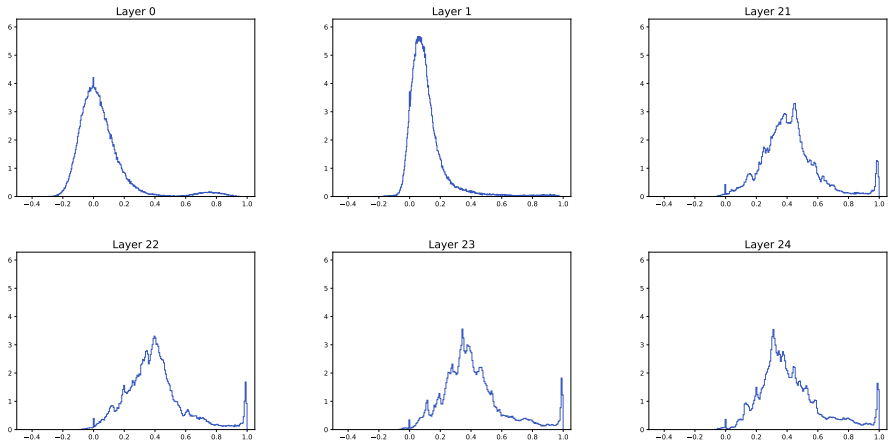
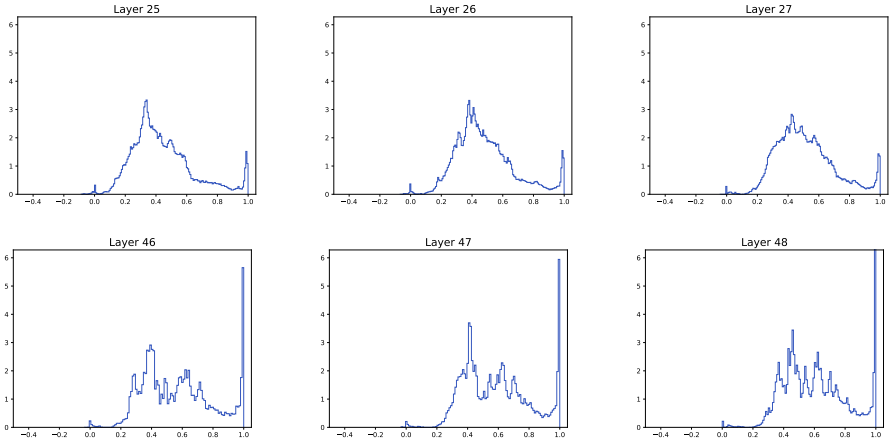


Figure 1. Histogram of $\{\langle x_i(t), x_j(t)\rangle\}_{(i,j)\in[n]^2, i\neq j}$ at different layers t in the context of the trained ALBERT XLarge v2 model ([LCG+20] and https://huggingface.co/albert-xlarge-v2)[3], which has constant parameter matrices. Here we randomly selected a single prompt, which in this context is a paragraph from a random Wikipedia entry, and then generate the histogram of the pairwise inner products. We see the progressive emergence of clusters all the way to the 24th (and last) hidden layer (top), as evidenced by the growing mass at 1. If the number of layers is increased, up to 48 say, the clustering is further enhanced (bottom).



Remark 2.1 (Collective behavior)**.** *The dynamics* (SA) *have a strong resemblance to the vast literature on nonlinear systems arising in the modeling of collective behavior. In addition to the connection to the classical Kuramoto model describing synchronization of oscillators* [Kur75, ABV+05] *(made evident in Section 7.2), Transformers are perhaps most similar to the Krause model* [Kra00]

$$\dot{x}_i(t) = \sum_{j=1}^{n} a_{ij}(x_j(t) - x_i(t)), \qquad a_{ij} = \frac{\phi(\|x_i - x_j\|^2)}{\sum_{k=1}^{n} \phi(\|x_i - x_k\|^2)}.$$

which is non-symmetric in general ($a_{ij} \neq a_{ji}$), much like (2.3). When ϕ is compactly supported, it has been shown in [JM14] that the particles $x_i(t)$ assemble in several clusters as $t \to +\infty$. Other related models include those of Vicsek [VCBJ+95], Hegselmann-Krause [HK02] and Cucker-Smale [CS07]. All these models exhibit a clustering behavior under various assumptions (see [MT14, Tad23] and the references therein). Yet, none of the opinion dynamics models discussed above contain parameters appearing within the nonlinearity as in (SA), whilst set on the sphere.

Remark 2.2 (Permutation equivariance)**.** *A function $f : (\mathbb{S}^{d-1})^n \to (\mathbb{S}^{d-1})^n$ is permutation equivariant if $f(\pi X) = \pi(f_1(X), \dots, f_n(X))$ for any $X \in (\mathbb{R}^d)^n$ and for any permutation $\pi \in \mathbf{S}_n$ of n elements. Otherwise put, if we permute the input X, then the output $f(X)$ is permuted in the same way. Given $t > 0$, the Transformer (SA), mapping $(x_i(0))_{i\in[n]} \mapsto (x_i(t))_{i\in[n]}$, is permutation-equivariant on $(\mathbb{S}^{d-1})^n$.*

2.3. **Toward the complete Transformer.** There are a couple of additional mechanisms used in practical implementations that we do not explicitly address or use in this study. The mathematical analysis of these mechanisms remains open.

2.3.1. *Multi-headed attention.* Practical implementations spread out the computation of the self-attention mechanism at every t through a sequence of *heads*, leading to the so-called *multi-headed self attention*. This consists in considering the following modification to (SA):

$$\dot{x}_i(t) = \mathbf{P}^{\perp}_{x_i(t)} \left(\sum_{h=1}^{H} \sum_{j=1}^{n} \frac{e^{\beta\langle Q_h(t)x_i(t), K_h(t)x_j(t)\rangle}}{Z_{\beta,i,h}(t)} V_h(t)x_j(t) \right) \tag{2.7}$$

where $Z_{\beta,i,h}(t)$ is defined as in (2.4) for the matrices $Q_h(t)$ and $K_h(t)$. The integer $H \geqslant 1$ is called the number of heads[4].

The introduction of multiple heads also allows for drawing some interesting parallels with the literature on feed-forward neural networks, such as ResNets (2.1). Considerable effort has been expended to understand 2-layer neural networks with width tending to $+\infty$; more precisely, consider (2.1) with $L = 1$, $w \in \mathbb{R}^{d\times\ell}$, $a \in \mathbb{R}^{\ell\times d}$, and $\ell \to +\infty$. The infinite-width limit for Transformers is in fact very natural, as it is realized by stacking an arbitrary large number of heads: $H \to +\infty$. Hence, the same questions as for 1-hidden layer neural networks may be asked: for instance the question of universal approximation, in the vein of [Cyb89, Bar93].

2.3.2. *Feed-forward layers.* The complete Transformer dynamics combines all of the above mechanisms with a feed-forward layer; in the discrete-time context, this is actually done by using a Lie-Trotter splitting scheme for

$$\dot{x}_i(t) = \mathbf{P}^{\perp}_{x_i(t)} \left(\sum_{h=1}^{H} \sum_{j=1}^{n} \frac{e^{\beta\langle Q_h(t)x_i(t), K_h(t)x_j(t)\rangle}}{Z_{\beta,i,h}(t)} V_h(t)x_j(t) + w(t)\sigma(a(t)x_i(t) + b(t)) \right), \tag{2.8}$$

[4]In practical implementations, H is a divisor of d, and the query and key matrices $Q_h(t)$ and $K_h(t)$ are $\frac{d}{H} \times d$ rectangular. This allows for further parallelization of computations and increased expressiveness. For mathematical purposes, we focus on working with arbitrary integers H, and square weight matrices Q_h and K_h.

where $w(t), a(t), b(t)$ and σ are all as in (2.2). The interested reader is referred to [LLH+20, PH22] for all the details[5]. The feed-forward layers (convolutional layers can alternatively be considered) are of critical importance in applications and drive the existing results on approximation properties of Transformers [YBR+19]. Nevertheless, the analysis of this model is beyond the scope of our current methods.

3. Measure to measure flow map

An important aspect of Transformers is that they are not hard-wired to take into account the order of the input sequence, contrary to other architectures used for natural language processing such as recurrent neural networks. In these applications, each token $x_i(0) \in \mathbb{R}^d$ contains not only a word embedding $w_i \in \mathbb{R}^d$, but also an additional *positional encoding* (we postpone a discussion to Remark 3.2) which allows tokens to also carry their position in the input sequence. Therefore, an input sequence is perfectly encoded as a *set* of tokens $\{x_1(0), \dots, x_n(0)\}$, or equivalently as the empirical measure of its constituent tokens $\frac{1}{n}\sum_{i=1}^n \delta_{x_i(0)}$. Recall that the output of a Transformer is also a probability measure, namely $\frac{1}{n}\sum_{i=1}^n \delta_{x_i(t)}$, albeit one that captures the likelihood of the next token. As a result, one can view Transformers as flow maps between probability measures[6] on $\mathbb{S}^{d-1}$. To describe this flow map, we appeal to the continuity equation, which governs precisely the evolution of the empirical measure of particles subject to dynamics. This perspective is already present in [SABP22], the only modification here being that we add the projection on the sphere arising from layer normalization.

After introducing the continuity equation in Section 3.1, we show that a particular interaction energy functional, which is maximized at any point mass, increases along solutions thereof in Section 3.2. Motivated by this monotonicity property, in Section 3.3 we propose an illustrative modified model which has the nice property of being a Wasserstein gradient flow for this energy. Finally, in Section 3.4, we demonstrate that the original equation presented in Section 3.1 is itself a gradient flow for the same energy, upon changing the metric underlying the definition of the gradient.

3.1. The continuity equation.

The vector field driving the evolution of a single particle in (SA) clearly depends on all n particles. In fact, one can equivalently rewrite the dynamics as

$$\dot{x}_i(t) = \mathcal{X}[\mu(t)](x_i(t)) \tag{3.1}$$

for all $i \in [n]$ and $t \geqslant 0$, where

$$\mu(t, \cdot) = \frac{1}{n}\sum_{i=1}^n \delta_{x_i(t)}(\cdot)$$

is the empirical measure, while the vector field $\mathcal{X}[\mu] : \mathbb{S}^{d-1} \to \mathrm{T}\mathbb{S}^{d-1}$ reads

$$\mathcal{X}[\mu](x) = \mathbf{P}_x^{\perp}\left(\frac{1}{Z_{\beta,\mu}(x)}\int e^{\beta\langle x,y\rangle} y \,\mathrm{d}\mu(y)\right) \tag{3.2}$$

[5] and lines 123–130 in [Ope24] for some relevant source code.

[6] See [DBPC19, VBC20, ZB21] for further related work on neural networks acting on probability measures.